

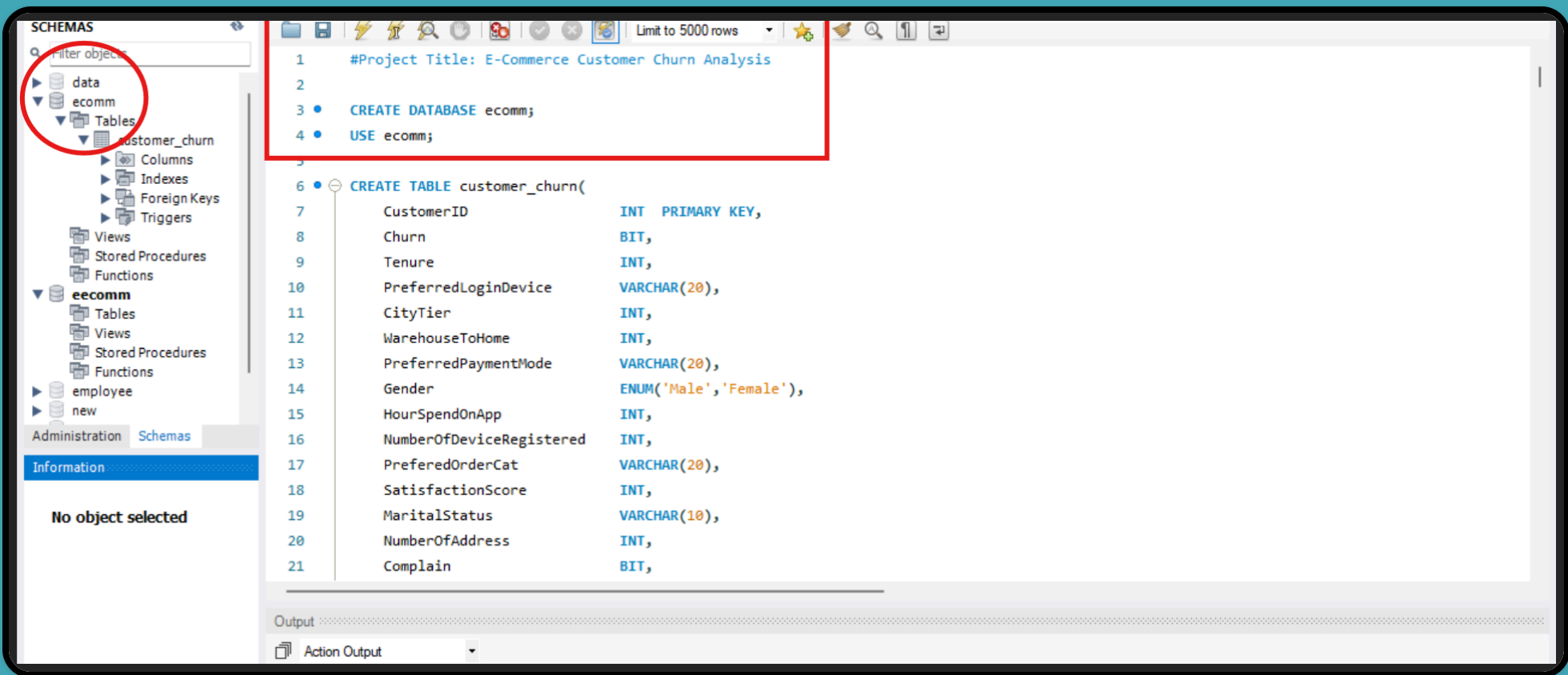
E-Commerce Customer Churn Analysis

DOCUMENTATION

part 1- Creating and Organising Dataset

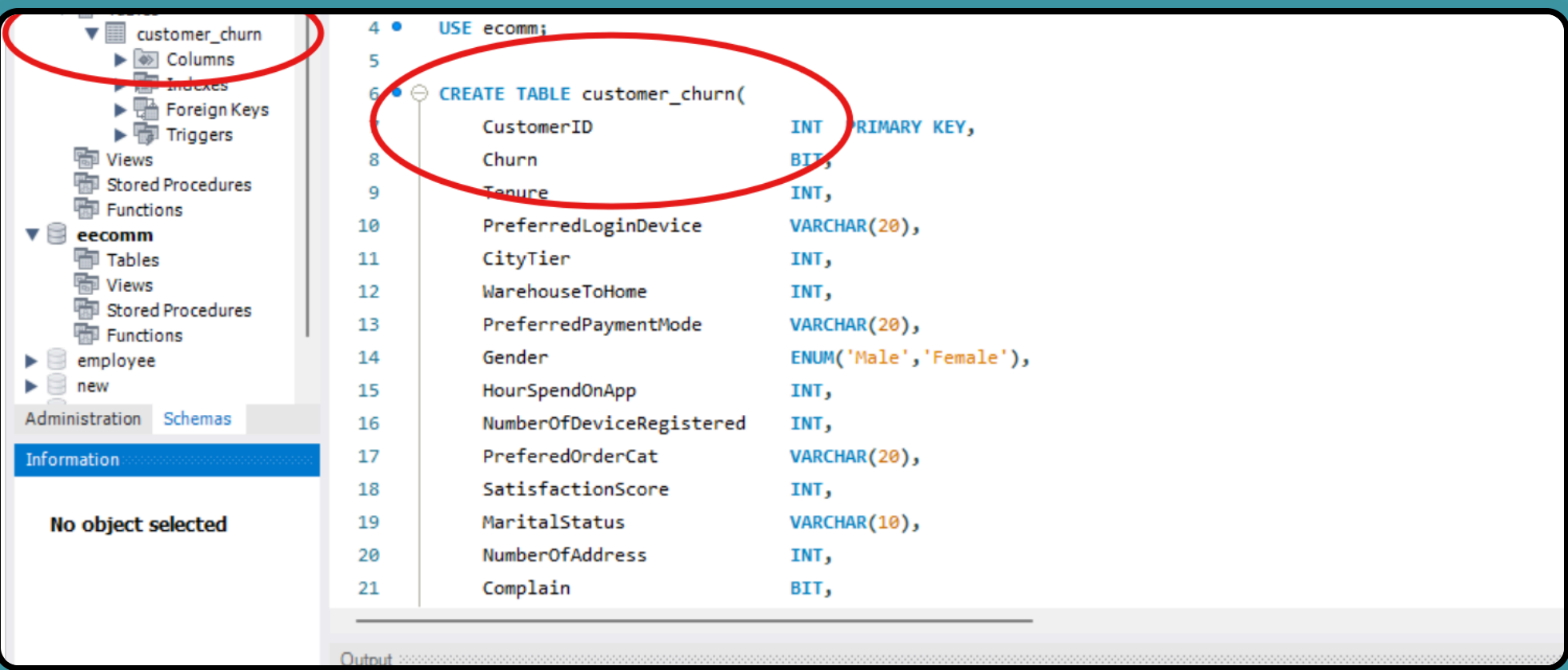
1-Create Database

Created Database called "ecomm" used to store, organize, manage, and retrieve data efficiently. It helps maintain data accuracy and consistency while reducing duplication and making information easier to access. A well-designed database also provides security, easy data management, and efficient analysis, which are important for making reliable business decisions



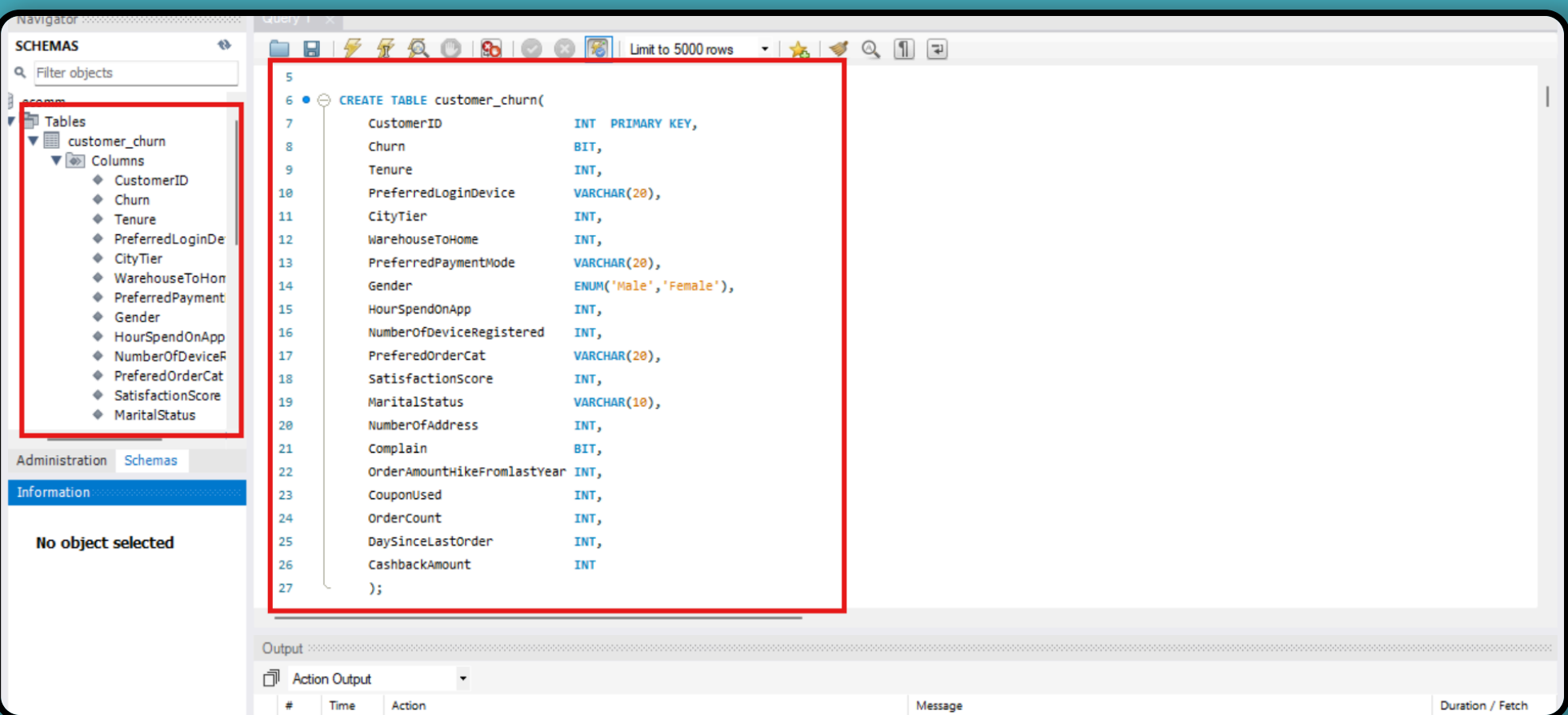
2-Creating the Customer_Churn Table

A table named Customer Churn was created along with appropriate database with the required columns, data types, and keys. Proper constraints and key relationships were defined to maintain data integrity, uniqueness, and consistency within the table. by using the "CREATE TABLE" statement. The table was designed to store customer-related information required for the analysis. Creating the table provided a structured way to organize, manage, and analyze the customer churn dataset.



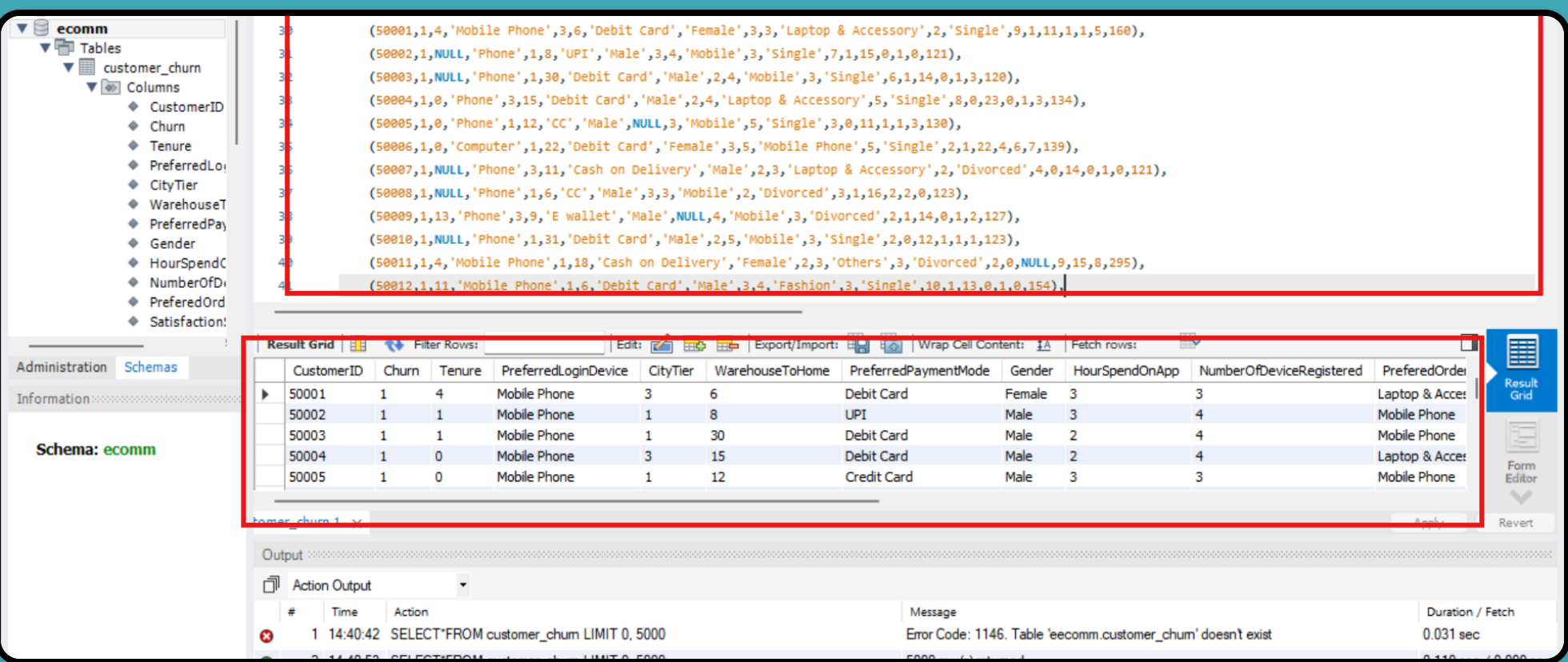
3-Columns

Columns-CustomerID(INTPRIMARY KEY),Churn(BIT), Tenure (INT), PreferredLoginDevice (VARCHAR(20),CityTier (INT), WarehouseToHome (INT),PreferredPaymentMode (VARCHAR(20), Gender(ENUM('Male', 'Female')),HourSpendOnApp (INT),NumberOfDeviceRegistered(INT), PreferredOrderCat (VARCHAR(20), SatisfactionScore (INT), MaritalStatus(VARCHAR(10), NumberOfAddress(INT), Complain (BIT),OrderAmountHikeFromlastYear (INT),CouponUsed (INT),OrderCount (INT),DaySinceLastOrder (INT),CashbackAmount (INT)



4-Dataset Import

The dataset was downloaded and the values were inserted into the appropriate table using the "INSERT" statement. The values were placed in the correct columns and maintained the required format for further analysis.



E-Commerce Customer Churn Analysis

DOCUMENTATION

part 2-Data Cleaning

1: Handling Missing Values(Imputation with Mean)

The first step of data cleaning involved handling missing values in the columns Warehouse to Home, Hours Spent on App, Order Amount Hike from Last Year, and Days Since Last Order. The missing values were imputed using the mean value of each respective column, calculated through inner queries, and the results were rounded off to the nearest integer. Using inner queries allowed the average values to be calculated dynamically from the existing data and then used within the main query to update only the required missing records. This approach makes the query more organized, reduces the need for separate calculations, and ensures that the imputed values are based on the actual dataset. Handling missing values in this way helps maintain data consistency, prevents loss of records, and improves the reliability of the dataset for further analysis.

```
654 #Q-1 Handling Missing Values and Outliers:Impute mean for the following columns,WarehouseToHome
655 #DaysSinceLastOrder
656
657 • USE ecomm;
658 • UPDATE customer_churn SET WarehouseToHome =
659   (SELECT ROUND(AVG(Z.WarehouseToHome)) FROM (SELECT WarehouseToHome FROM customer_churn)AS Z)
670 WHERE WarehouseToHome IS NULL;
671 • SELECT * FROM customer_churn;
672
673
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	1	Phone	1	8	UPI	Male	3	4
50003	1	1	Phone	1	30	Debit Card	Male	2	4
50004	1	0	Phone	3	15	Debit Card	Male	2	4
50005	1	0	Phone	1	12	CC	Male	3	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	1	Phone	3	11	Cash on Delivery	Male	2	3

```
70 WHERE WarehouseToHome IS NULL;
71 • SELECT * FROM customer_churn;
72
73 #IMPUTE HourSpentOnApp
74 • UPDATE customer_churn SET HourSpentOnApp=
75   (SELECT ROUND(AVG(Z.HourSpentOnApp)) FROM (SELECT HourSpentOnApp FROM Customer_churn)AS Z)
76 WHERE HourSpentOnApp IS NULL;
77
78
79
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	1	Phone	1	8	UPI	Male	3	4
50003	1	1	Phone	1	30	Debit Card	Male	2	4
50004	1	0	Phone	3	15	Debit Card	Male	2	4
50005	1	0	Phone	1	12	CC	Male	3	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	1	Phone	3	11	Cash on Delivery	Male	2	3

2-Handling Missing Values(Imputation with Mode)

The next step in data cleaning was to handle missing values using mode imputation. Missing values in the columns Tenure, Coupon Used, and Order Count were replaced with the mode, which is the most frequently occurring value in each respective column. The mode was calculated within the SQL query using inner queries(GROUP BY,COUNT,ORDER BY & LIMIT) and the calculated value was then used to update the missing records. Using mode is suitable for these columns because it preserves the most commonly occurring value in the dataset and helps maintain consistency without removing records. This approach also allows the missing values to be handled dynamically based on the existing data.

```
887 WHERE DaySinceLastOrder IS NULL;
888
889 #Impute mode for the following columns: Tenure, CouponUsed, OrderCount.
890
891 #IMPUTE Tenure WITH MODE
892 UPDATE customer_churn SET Tenure=
893   (SELECT Tenure FROM (SELECT Tenure FROM Customer_churn WHERE Tenure IS NOT NULL
894     GROUP BY Tenure ORDER BY COUNT(*) DESC LIMIT 1)AS Z)
895 WHERE Tenure IS NULL;
896
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	1	Phone	1	8	UPI	Male	3	4
50003	1	1	Phone	1	30	Debit Card	Male	2	4
50004	1	0	Phone	3	15	Debit Card	Male	2	4
50005	1	0	Phone	1	12	CC	Male	3	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	1	Phone	3	11	Cash on Delivery	Male	2	3

```
94 GROUP BY Tenure ORDER BY COUNT(*) DESC LIMIT 1)AS Z)
95 WHERE Tenure IS NULL;
96
97 #IMPUTE CouponUsed WITH MODE
98 UPDATE Customer_churn SET CouponUsed=
99   (SELECT CouponUsed FROM(SELECT CouponUsed FROM Customer_churn WHERE CouponUsed IS NOT NULL
100     GROUP BY CouponUsed ORDER BY COUNT(*) DESC LIMIT 1)AS Z)
101 WHERE CouponUsed IS NULL;
102 • SELECT *FROM customer_churn;
103
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	1	Phone	1	8	UPI	Male	3	4
50003	1	1	Phone	1	30	Debit Card	Male	2	4
50004	1	0	Phone	3	15	Debit Card	Male	2	4
50005	1	0	Phone	1	12	CC	Male	3	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	1	Phone	3	11	Cash on Delivery	Male	2	3

3-Syntax Used for Data cleaning

Data cleaning and imputation was performed using "SELECT", "UPDATE", "SET", and "WHERE" clauses along with "IS NULL". "SELECT" was used to calculate and retrieve values, "UPDATE" and "SET" were used to replace missing values, and "WHERE" with "IS NULL" was used to identify and update only the required records. These clauses helped clean the data efficiently and accurately.

```
5663 #required: warehouseToHome, hourSpentOnApp, OrderAmountHikeFromLastYear,DaysSinceLastOrder.
5664
5665 #IMPUTE_WarehouseToHome
5666 USE ecomm;
5667 • UPDATE Customer_churn SET WarehouseToHome=
5668   (SELECT ROUND(AVG(Z.warehouseToHome)) FROM (SELECT WarehouseToHome FROM Customer_churn)AS Z)
5669 WHERE WarehouseToHome IS NULL;
5670
5671 # IMPUTE_hourSpentOnApp
5672 UPDATE customer_churn SET HourSpentOnApp=
5673   (SELECT ROUND(AVG(Z.HourSpentOnApp)) FROM (SELECT HourSpentOnApp FROM Customer_churn)AS Z)
5674 WHERE HourSpentOnApp IS NULL;
5675
5676 #IMPUTE OrderAmountHikeFromLastYear
5677 UPDATE Customer_churn SET OrderAmountHikeFromLastYear=
5678   (SELECT ROUND(AVG(Z.OrderAmountHikeFromLastYear)) FROM (SELECT OrderAmountHikeFromLastYear FROM Customer_churn) AS Z)
5679 WHERE OrderAmountHikeFromLastYear IS NULL;
5680
5681
5682 #IMPUTE DAYSSINCE LAST ORDER
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	1	Phone	1	8	UPI	Male	3	4
50003	1	1	Phone	1	30	Debit Card	Male	2	4
50004	1	0	Phone	3	15	Debit Card	Male	2	4
50005	1	0	Phone	1	12	CC	Male	3	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	1	Phone	3	11	Cash on Delivery	Male	2	3

4-Handle outliers

As part of handling outliers, rows with values greater than 100 in the Warehouse to Home column were identified and deleted. These values were considered extreme compared with the expected range of the data and could negatively affect the accuracy of further analysis. Removing such outliers helps reduce the impact of abnormal values, improve data quality, and ensure that the dataset is more suitable for meaningful analysis.

```
5710 WHERE OrderCount IS NULL;
5711
5712 #Handle outliers in the 'WarehouseToHome' column by deleting rows where the values are greater than 100.
5713 • DELETE FROM Customer_churn
5714 WHERE WarehouseToHome >100;
5715
5716
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
51307	0	5	Mobile Phone	3	6	E wallet	Female	2	3
51308	0	10	Mobile Phone	1	16	Debit Card	Female	2	1
51309	0	0	Phone	3	14	UPI	Female	2	3
51311	0	17	Phone	1	16	CC	Male	2	3
51312	0	0	Phone	3	16	CC	Male	2	3
51315	0	12	Computer	1	15	Debit Card	Male	2	3

5-Dealing with Inconsistencies

Inconsistent values in the dataset were standardized to ensure uniformity. The value "phone" in the Preferred Login Device column and "mobile" in the Preferred Order Category column were replaced with "mobile phone". This helped maintain consistent data values and improved the accuracy of further analysis.

SQL Syntax Used: The "UPDATE", "SET", "REPLACE", and "WHERE" clauses, along with nested subqueries, were used to standardize inconsistent values. "REPLACE" changed the specified values to "mobile phone", while "WHERE" ensured that only the required records were updated. Nested subqueries helped perform the required operations within the main query efficiently.

```
5718 #Replace occurrences of "Phone" in the 'PreferredLoginDevice' column and
5719 #"Mobile" in the 'PreferredOrderCat' column with "Mobile Phone" to ensure uniformity.
5720
5721 UPDATE Customer_churn SET PreferredLoginDevice
5722   REPLACE(PreferredLoginDevice , 'Phone', 'Mobile Phone')
5723 WHERE PreferredLoginDevice='Phone';
5724
5725
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	1	Mobile Phone	1	8	UPI	Male	3	4
50003	1	1	Mobile Phone	1	30	Debit Card	Male	2	4
50004	1	0	Mobile Phone	3	15	Debit Card	Male	2	4
50005	1	0	Mobile Phone	1	12	CC	Male	3	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	1	Mobile Phone	3	11	Cash on Delivery	Male	2	3

```
5713
5714 • UPDATE Customer_churn SET PreferredLoginDevices
5715   REPLACE(PreferredLoginDevice , 'Phone', 'Mobile Phone')
5716 WHERE PreferredLoginDevices='Phone';
5717
5718 UPDATE Customer_churn SET PreferredOrderCats
5719   REPLACE(PreferredOrderCat, 'Mobile','Mobile Phone')
5720 WHERE PreferredOrderCats='Mobile';
5721
```

PreferredPaymentMode	Gender	HourSpentOnApp	NumberOfDevicesRegistered	PreferredOrderCat	SatisfactionScore	MaritalStatus	NumberOfAddress	Complain	OrderCount
Mobile Phone	Male	2	4	Mobile Phone	3	Single	6	1	14
Mobile Phone	Male	2	4	Mobile Phone	3	Single	8	0	23
Mobile Phone	Male	3	3	Mobile Phone	5	Single	3	0	11
Mobile Phone	Female	3	5	Mobile Phone	3	Single	2	1	22
Laptop & Accessory	Male	2	3	Laptop & Accessory	4	Divorced	4	0	14
Mobile Phone	Male	3	3	Mobile Phone	2	Divorced	3	1	16
Mobile Phone	Male	3	4	Mobile Phone	2	Divorced	2	1	14

part 3-Data Exploration and Analysis I

The column names were corrected using the "ALTER TABLE ... RENAME COLUMN" statement. Preferred Order Cat was renamed to Preferred Order Cat, and Hours Spent on App was renamed to Hours Spent on App to ensure proper spelling and consistency in the database.



The screenshot shows the DBeaver SQL editor interface. On the left, the 'Schemas' pane displays a tree view of the database structure, including 'ecommerce' and its tables. The main editor area contains a SQL query that checks for null values in the 'churn' and 'complain' columns. The query is as follows:

```

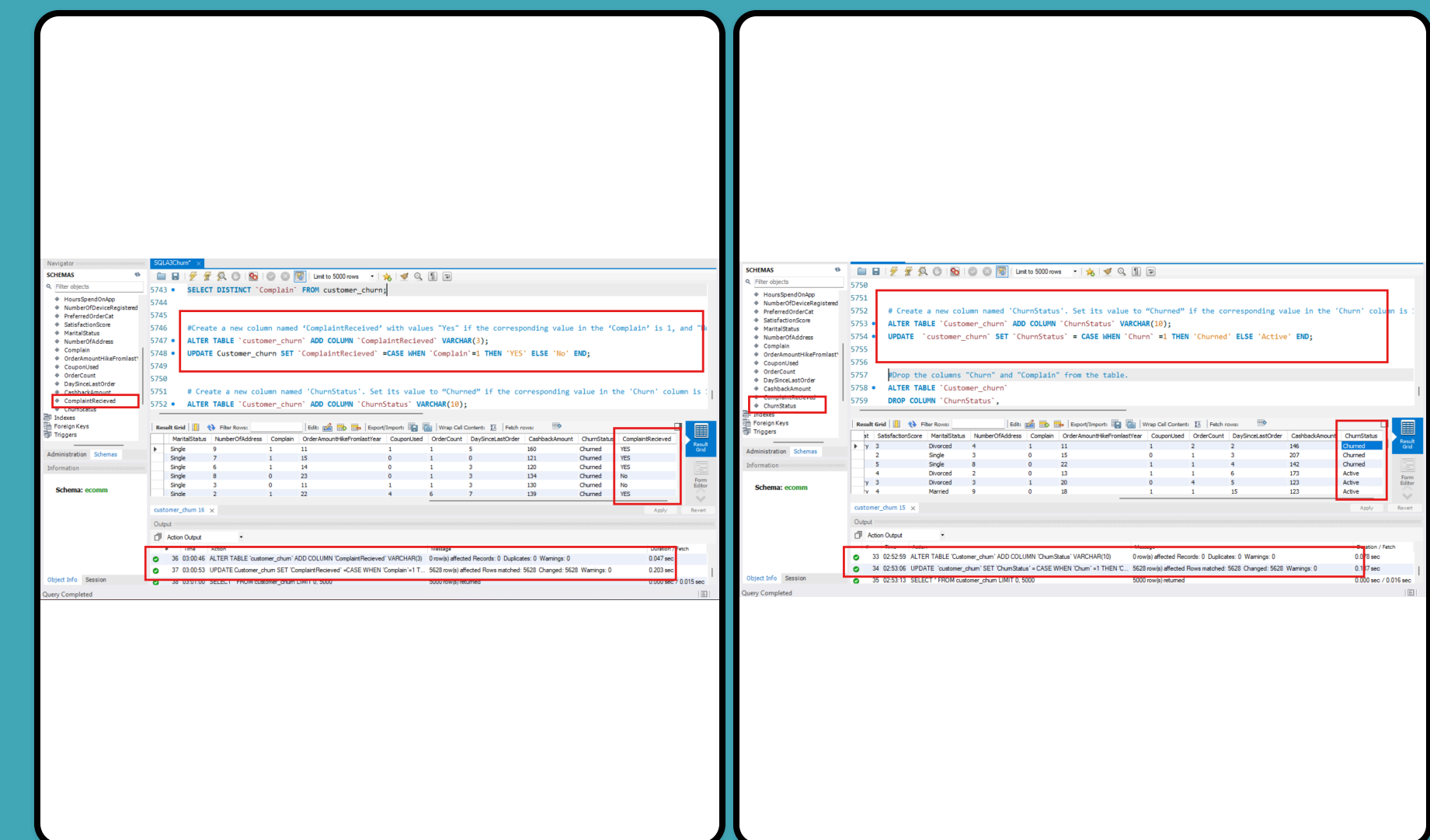
--(Verified that the churn and Complain column contain no null values and only expected values 0 and 1
--Before creating the corresponding status columns)
SELECT DISTINCT 'churn' FROM 'customer_churn' WHERE 'churn' NOT IN (0,1);
SELECT DISTINCT 'complain' FROM customer_churn;

```

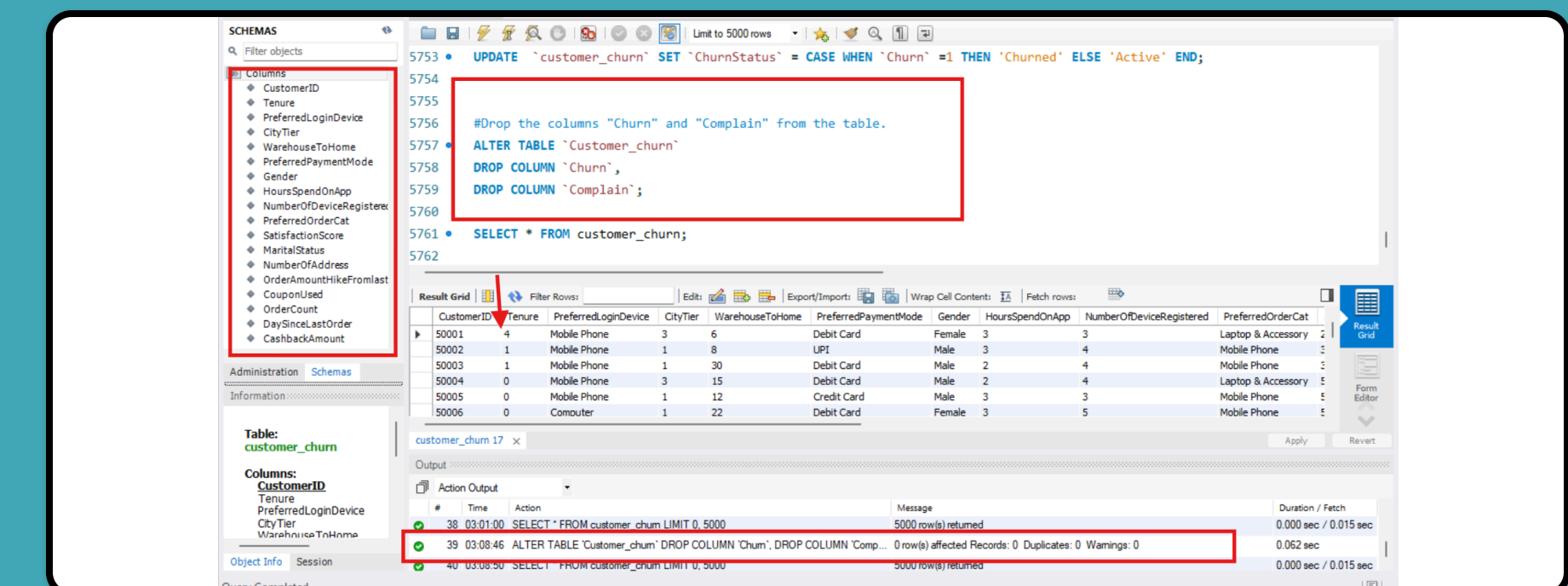
Below the query, the 'Results' pane shows the output of the query. It contains two rows: 'churn' and 'complain', each with a value of 0. The 'churn' row is highlighted in red.

The 'Query Output' pane at the bottom shows the execution details of the query. It indicates that the query was executed successfully and returned 2 rows.

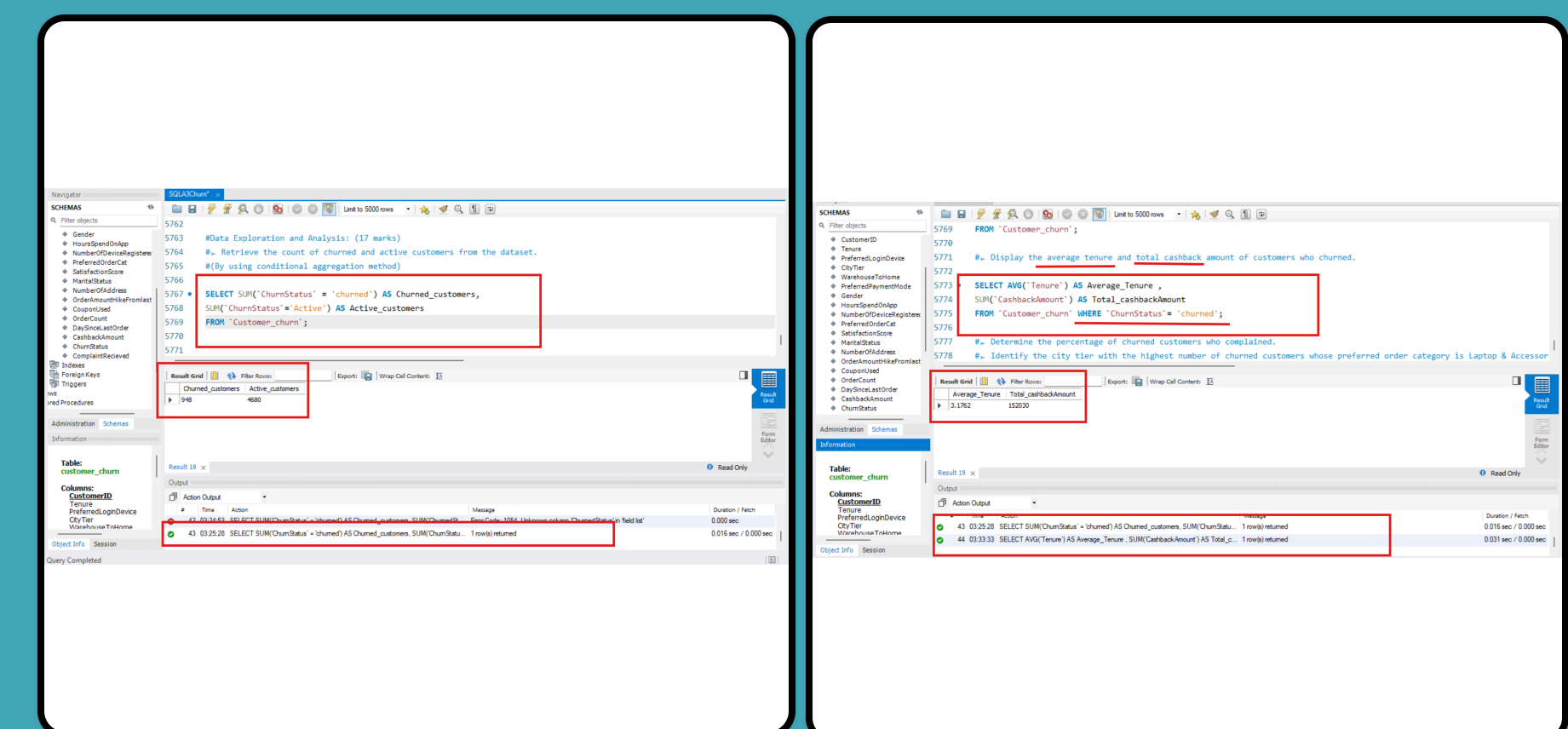
Two corresponding status columns, Complaint Received and Churn Status, were created using "ALTER TABLE ... ADD COLUMN" and updated with a "CASE" statement. Complaint Received displays "Yes" when the Complaint value is "1" and "No" otherwise, while Churn Status displays "Churned" when the Churn value is "1" and "Active" otherwise. These conditional status columns make the data easier to understand and analyze by clearly representing complaint and customer churn status. The ADD COLUMN and UPDATE method was used to create and populate the new status columns because it allows the existing values to be converted into meaningful labels while keeping the new columns independent from the original columns. Unlike a trigger, which mainly handles future inserted records, and a generated column, which remains dependent on the original column, this method allows the new values to be stored permanently. This is especially suitable because the original Churn and Complaint columns need to be dropped after the data-cleaning process.



After creating and populating the corresponding status columns, the original "Churn" and "Complaint" columns were removed to eliminate redundant data and maintain a clean, organized dataset. The new status columns preserve the required information in a more meaningful and readable format.



The count of churned and active customers was retrieved using the conditional aggregation method. "SUM()" was used with conditions to count customers whose "Churn Status" was 'Churned' and 'Active', providing the total number of customers in each category. The query uses AVG() to calculate the average tenure and SUM() to calculate the total cashback amount for customers who have churned. The WHERE condition filters the dataset to include only customers whose Churn Status is 'Churned', ensuring that the calculations are based only on churned customers.



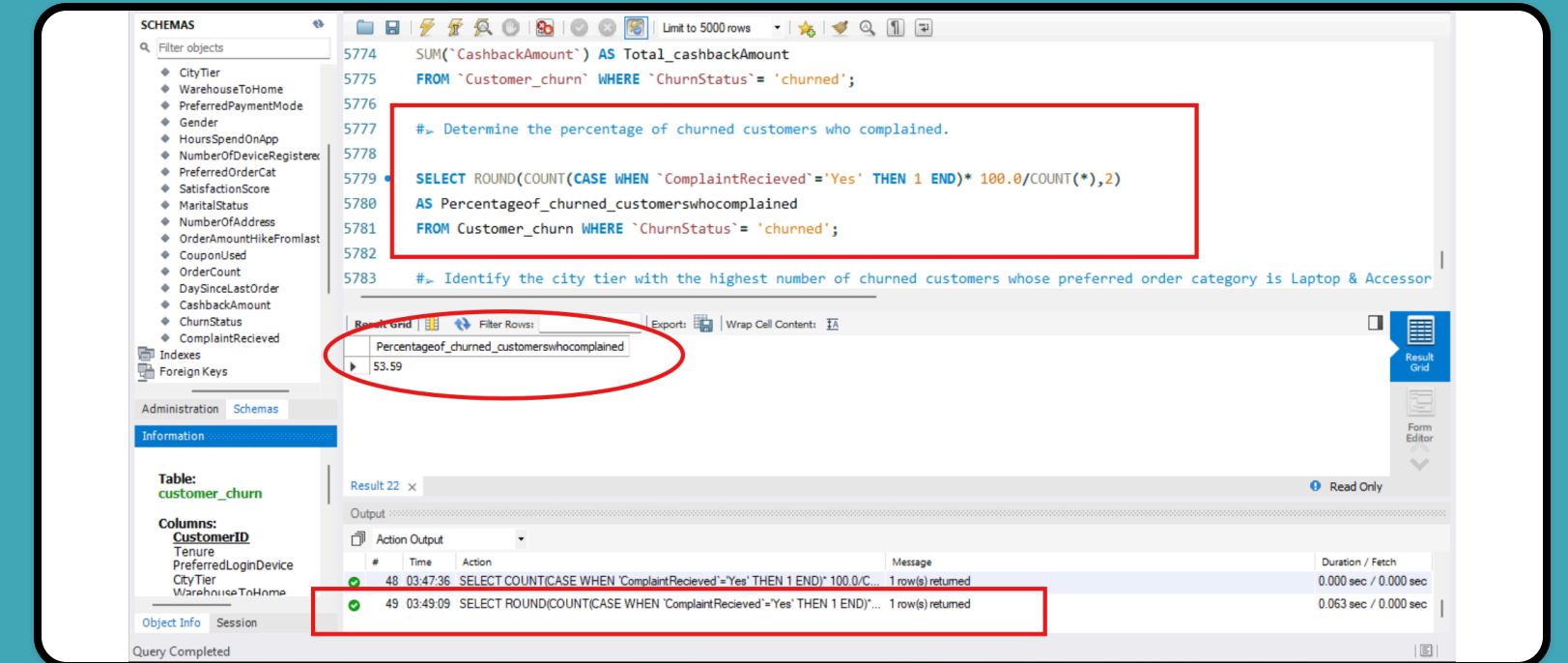
E-Commerce Customer Churn Analysis DOCUMENTATION

part 3-Data Exploration and Analysis II

1 :Percentage of Churned Customers Who Complained

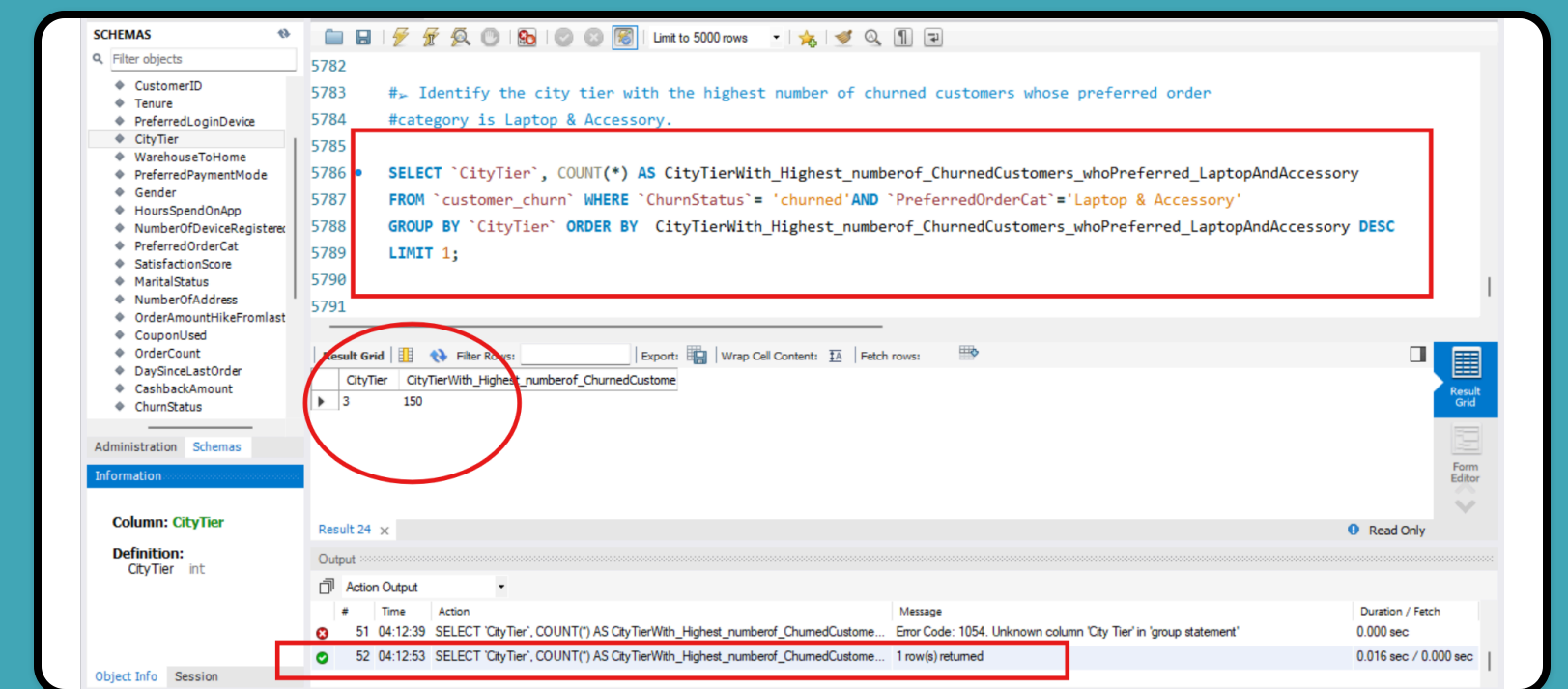
The percentage of churned customers who complained was calculated and rounded to two decimal places for better readability and understanding. This helps clearly represent the proportion of churned customers who submitted complaints and supports easier interpretation during data analysis.Cyntax used:SELECT ROUND(COUNT(CASE WHEN `ComplaintRecieved`=`Yes' THEN 1 END)* 100.0/COUNT(*),2)

AS Percentageof_churned_customerswhocomplained
FROM Customer_churn WHERE `ChurnStatus`=`churned`;



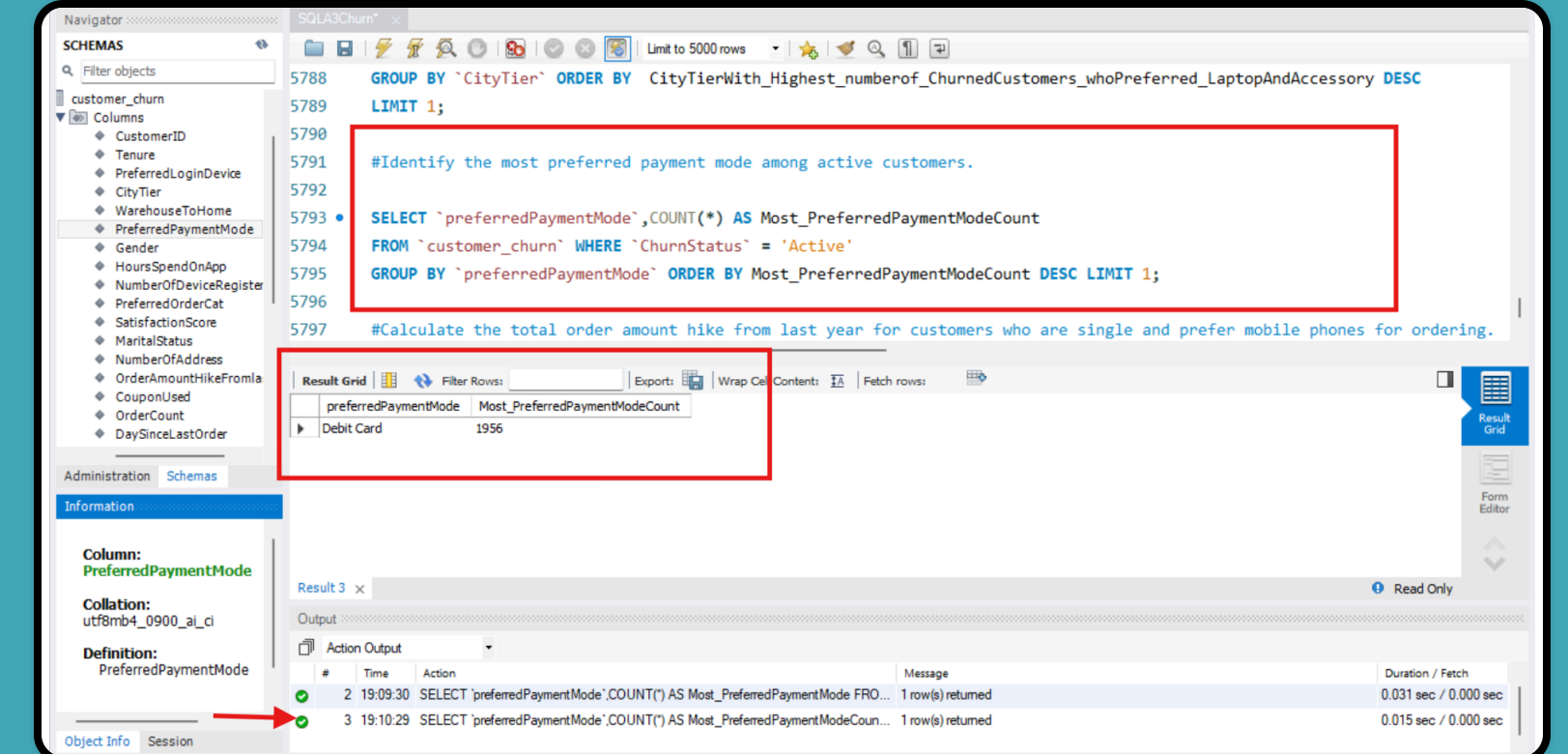
2 :Grouping & Counting City Tier

City Tier is a classification used to group cities based on factors such as population, size, economic development, and urbanization. In the query, two conditions are applied: Churn Status = 'Churned' to include only churned customers, and Preferred Order Cat = 'Laptop & Accessory' to include only customers who preferred the Laptop & Accessory category. The customers are then grouped by city tier and counted. The results are arranged in descending order so the city tier with the highest number of relevant churned customers appears first, and LIMIT 1 is used to display only the highest-ranking city tier.



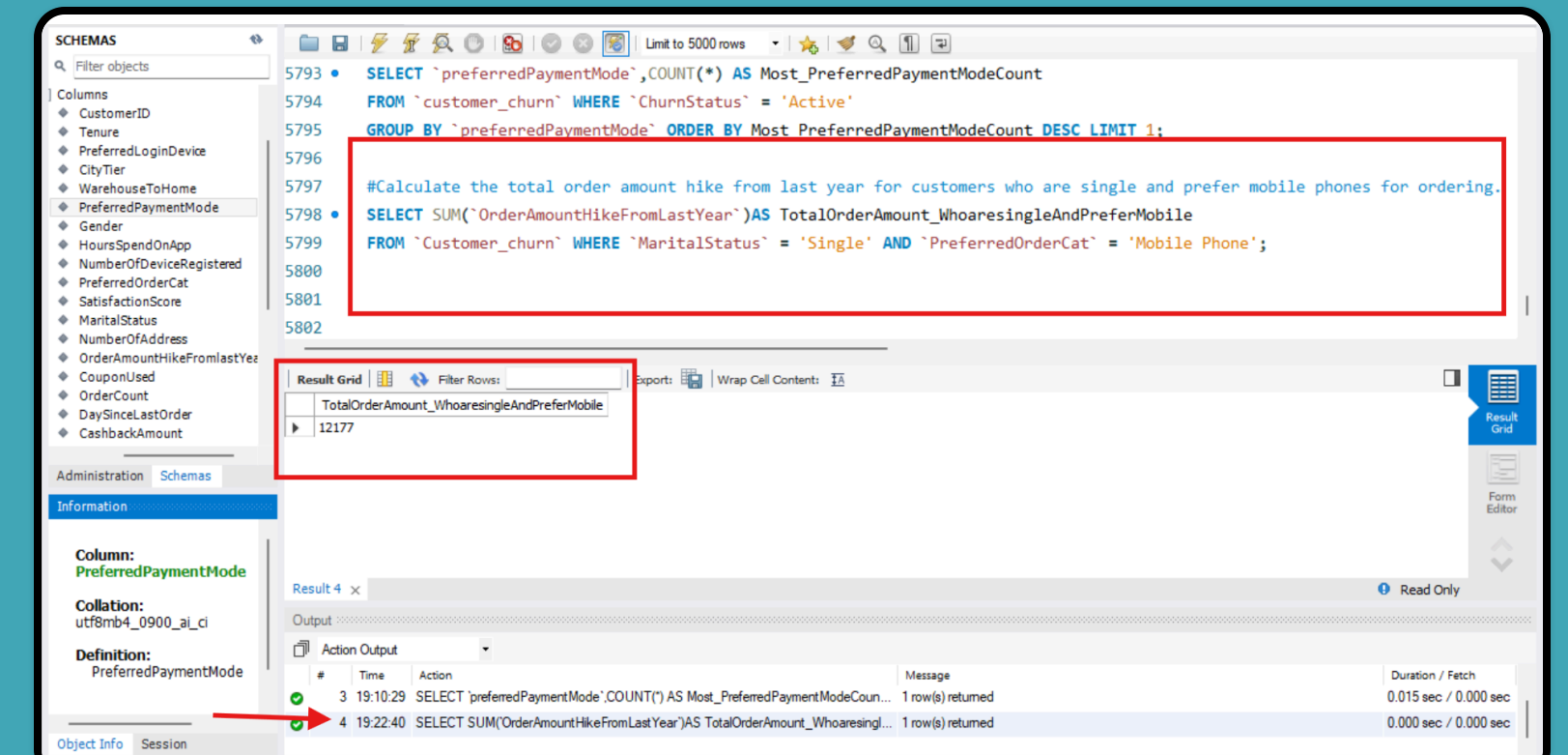
3 :Most Preferred Payment Mode Among Active Customers

The Preferred Payment Mode column was analyzed for active customers by filtering Churn Status as 'Active'. Each payment mode was grouped and counted to determine how frequently it was used. The results were arranged in descending order, and LIMIT 1 was used to display the most frequently preferred payment mode. The result shows that Debit Card is the most preferred payment mode among active customers, with 1,956 customers using it.



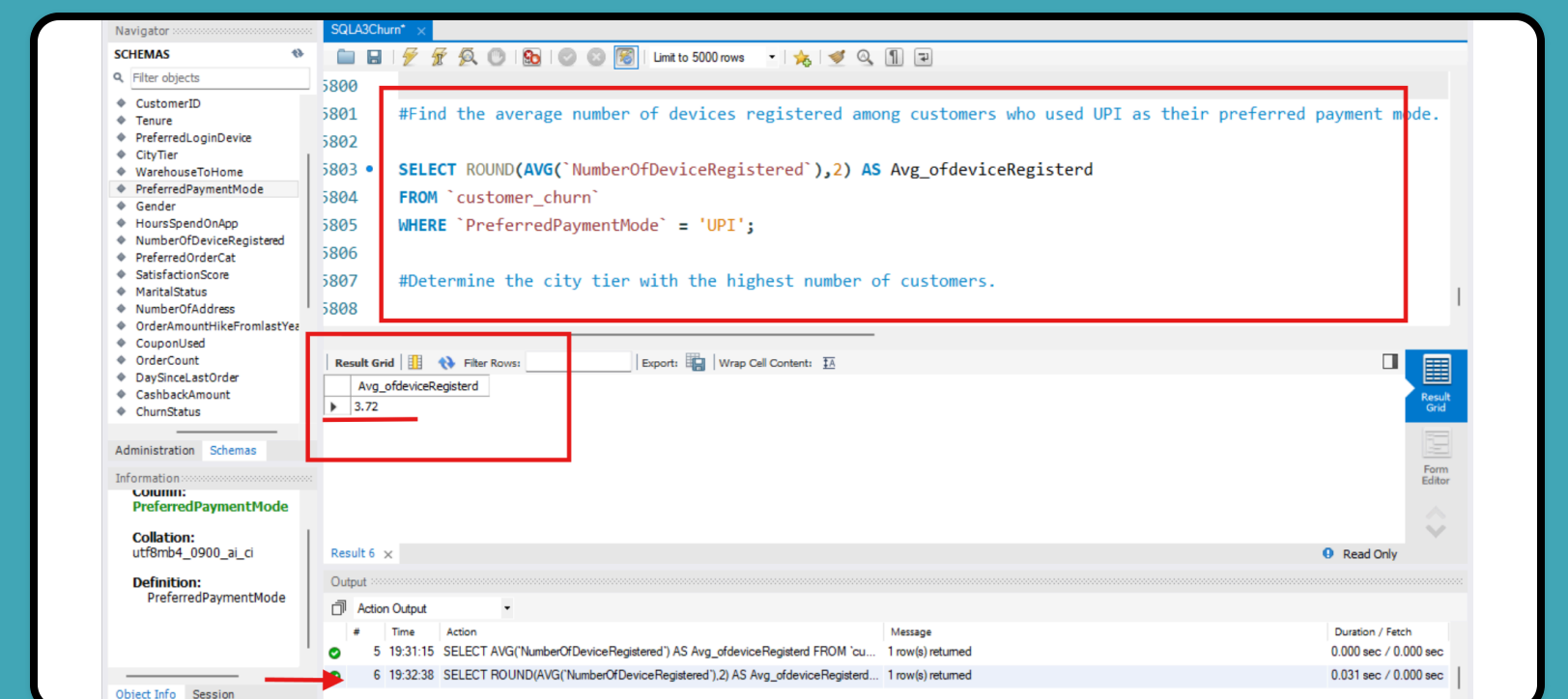
4 :Total Order Amount Hike from Last Year

A simple aggregate query using the SUM() function and WHERE conditions was used to calculate the total order amount hike from last year. The query filters the customers whose Marital Status is 'Single' and whose Preferred Order Category is 'Mobile Phone'. The SUM() function calculates the total value of Order Amount Hike from Last Year for these customers. The result obtained is 12,177. This query was used to determine the overall order amount increase for this specific group of customers.



5 :Average Number of Devices Registered Among UPI Users

The average number of devices registered among customers who use UPI as their preferred payment mode was calculated using the AVG() function with a WHERE clause. The ROUND() function was used to round the result to two decimal places for better readability. Before applying ROUND(), the result was 3.7174; after rounding, the final result was 3.72 devices.



E-Commerce Customer Churn Analysis

DOCUMENTATION

part 3-Data Exploration and Analysis III

1:City Tier with the Highest Number of Customers

The query was used to determine which city tier has the highest number of customers. The SELECT statement displays the city tier, while the COUNT() function calculates the number of customers in each tier. The FROM clause specifies the Customer Churn table, and GROUP BY groups the customers based on their city tier. ORDER BY DESC sorts the results from the highest customer count to the lowest, and LIMIT 1 displays only the city tier with the highest count. The result showed that City Tier 1 has the highest number of customers, with 3,666 customers.

```
SELECT ROUND(AVG('NumberDeviceRegistered'),2) AS Avg_ofdeviceRegistered
FROM 'customer_churn'

WHERE 'PreferredPaymentMode' = 'UPI';

#Determine the city tier with the highest number of customers.
SELECT 'CityTier', COUNT(*) AS HighestNumberOfCustomers
FROM 'customer_churn' GROUP BY 'CityTier' ORDER BY
HighestNumberOfCustomers DESC LIMIT 1;
```

CityTier	HighestNumberOfCustomers
1	3666

2:Gender with the Highest Number of Coupons Used

The query was used to identify the gender that utilized the highest number of coupons. The SELECT statement displays the gender, while the SUM() function calculates the total number of coupons used by each gender. The FROM clause specifies the Customer Churn table, and GROUP BY Gender groups the records according to gender. ORDER BY with DESC arranges the results from the highest total coupon usage to the lowest, and LIMIT 1 displays only the gender with the highest number of coupons used, as the question requires one result. The output showed that Male customers utilized the highest number of coupons, with a total of 5,629 coupons used.

```
#Identify the gender that utilized the highest number of coupons.
SELECT 'Gender', SUM(CouponUsed) AS HighestNumberOfCouponsUsed_Gender
FROM 'customer_churn' GROUP BY 'Gender'
ORDER BY HighestNumberOfCouponsUsed_Gender
DESC LIMIT 1;
```

Gender	HighestNumberOfCouponsUsed_Gender
Male	5629

3:Total Order Count for Credit Card Users with Maximum Satisfaction Score

The query was used to calculate the total order count for customers who prefer Credit Card as their payment mode and have the maximum satisfaction score. The SUM() function calculates the total order count, while the WHERE clause filters the records to include only customers using Credit Card. An inner query using MAX(Satisfaction_Score) identifies the highest satisfaction score in the dataset, ensuring that only customers with the maximum score are considered. The result showed a total order count of 1,122, and the maximum satisfaction score was 5.

```
#List the number of customers and the maximum hours spent on the app in each preferred order category.
SELECT 'PreferredOrderCat', COUNT(*) AS 'NumberOfCustomers', MAX('HoursSpentOnTheApp') AS MAX_HoursSpentOnTheApp
FROM 'customer_churn' GROUP BY 'PreferredOrderCat';

#List the preferred order category among customers who used more than 5 coupons.
SELECT 'CustomerID', 'PreferredOrderCat', 'CouponUsed' FROM 'customer_churn' WHERE CouponUsed > 5;
```

PreferredOrderCat	NumberOfCustomers	MAX_HoursSpentOnTheApp
Laptop & Accessory	202	5
Mobile Phone	276	5
Others	264	4
Fashion	826	5
Grocery	422	4

4:Average Satisfaction Score of Customers Who Complained

The query was used to calculate the average satisfaction score of customers who had complaints. The AVG() function calculates the average satisfaction score, while the ROUND() function rounds the result to two decimal places for clearer presentation. The FROM clause specifies the Customer Churn table, and the WHERE clause filters the records to include only customers whose Complaint Received status is Yes. The result showed that the average satisfaction score of customers who complained was 3.00.

```
What is the average satisfaction score of customers who have complained?
SELECT ROUND(AVG('SatisfactionScore'),2) AS AVG_ofSatisfactionScore_whoComplained
FROM 'customer_churn' WHERE 'ComplaintReceived' = 'Yes';

#List the preferred order category among customers who used more than 5 coupons.
SELECT 'CustomerID', 'PreferredOrderCat', 'CouponUsed' FROM 'customer_churn' WHERE CouponUsed > 5;
```

AVG_ofSatisfactionScore_whoComplained
3.00

5:Preferred Order Category Among Customers Using More Than Five Coupons

The query was used to list the preferred order categories of customers who used more than five coupons. The SELECT statement displays the Customer ID, Preferred Order Category, and Coupon Used values. The FROM clause specifies the Customer Churn table, while the WHERE clause filters the records to include only customers who used more than five coupons. The output shows each eligible customer along with their preferred order category and the number of coupons used.

```
SELECT 'CustomerID', 'PreferredOrderCat', 'CouponUsed' FROM 'customer_churn' WHERE CouponUsed > 5;

#List the preferred order category among customers who used more than 5 coupons.
SELECT 'CustomerID', 'PreferredOrderCat', 'CouponUsed' FROM 'customer_churn' WHERE CouponUsed > 5;

#List the top 3 preferred order categories with the highest average cashback amount.
SELECT 'PreferredOrderCat', ROUND(AVG('CashBackAmount'),2) AS AVG_CashBack FROM 'customer_churn'
GROUP BY 'PreferredOrderCat' ORDER BY AVG_CashBack DESC LIMIT 3;
```

CustomerID	PreferredOrderCat	CouponUsed
1001	Others	9
1002	Laptop & Accessory	6
1003	Fashion	11
1004	Fashion	7
1005	Fashion	12
1006	Fashion	35
1007	Laptop & Accessory	6
1008	Laptop & Accessory	6

6:Top Three Preferred Order Categories by Average Cashback

The query was used to identify the top three preferred order categories with the highest average cashback amount. The AVG() function calculates the average cashback amount for each order category, while the ROUND() function rounds the decimal values for easier interpretation. The FROM clause specifies the Customer Churn table, and GROUP BY groups the records according to the preferred order category. ORDER BY DESC sorts the categories from the highest average cashback to the lowest, and LIMIT 3 displays only the top three categories as required. The result showed Others, Grocery, and Fashion as the top three preferred order categories based on average cashback amount.

```
SELECT 'PreferredOrderCat', 'CouponUsed' FROM 'customer_churn' WHERE CouponUsed > 5;

#List the top 3 preferred order categories with the highest average cashback amount.
SELECT 'PreferredOrderCat', ROUND(AVG('CashBackAmount'),2) AS AVG_CashBack FROM 'customer_churn'
GROUP BY 'PreferredOrderCat' ORDER BY AVG_CashBack DESC LIMIT 3;

#List the preferred payment modes of customers whose average tenure is 18 months and have placed more than 500 orders.
SELECT 'PreferredOrderCat', ROUND(AVG('CashBackAmount'),2) AS AVG_CashBack FROM 'customer_churn'
GROUP BY 'PreferredOrderCat' ORDER BY AVG_CashBack DESC LIMIT 3;
```

PreferredOrderCat	AVG_CashBack
Others	24.46
Grocery	26.24
Fashion	23.46

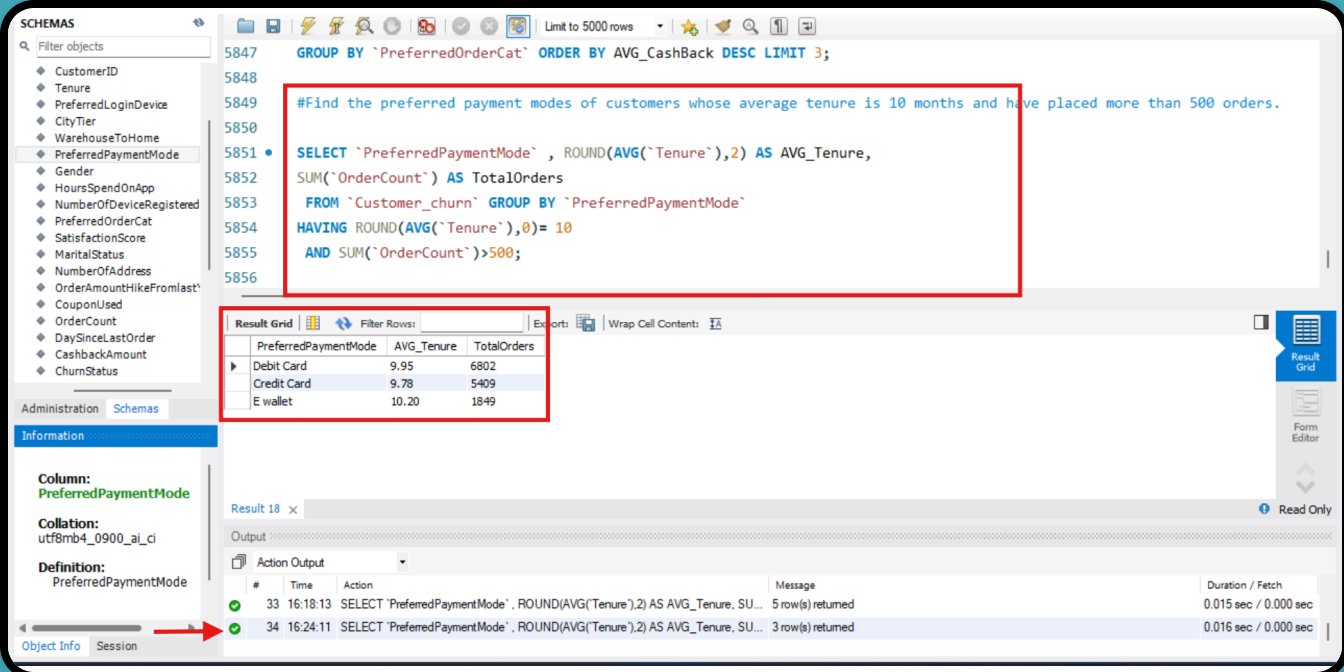
E-Commerce Customer Churn Analysis

DOCUMENTATION

part 4 -Customer Categorization

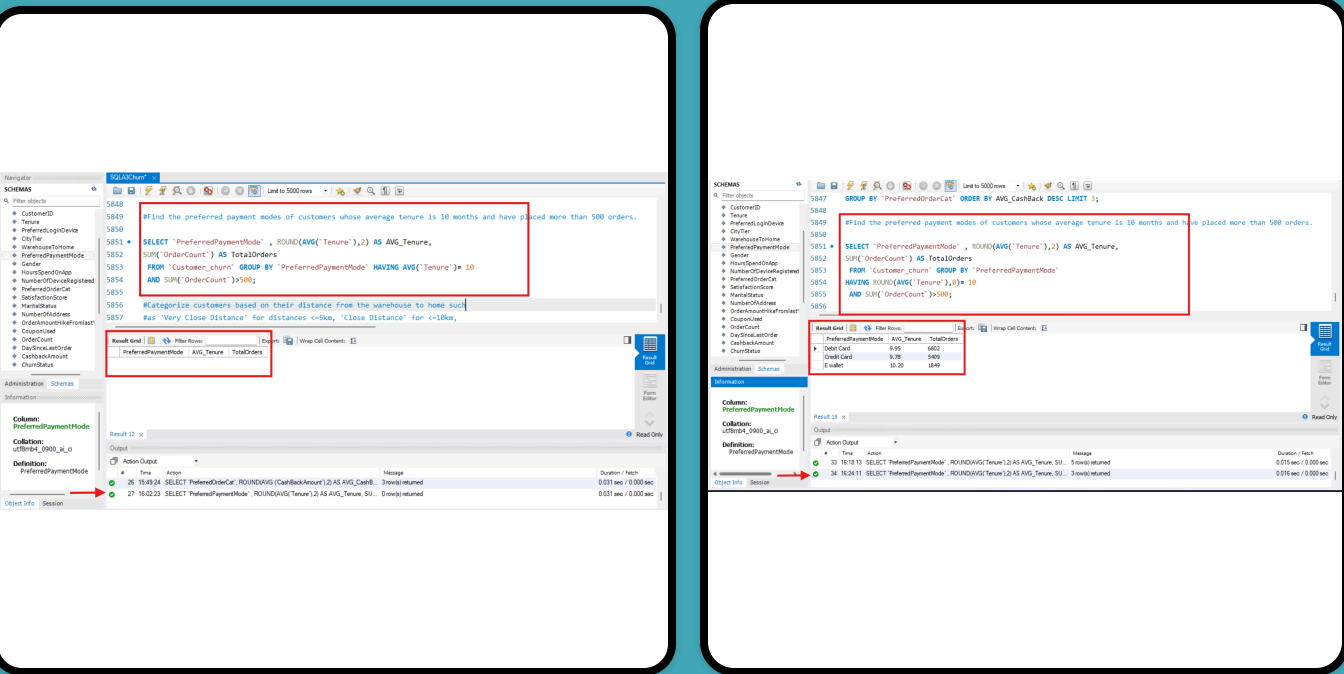
1:Preferred Payment Mode with Average Tenure of 10 Months and More Than 500 Orders

The query was used to identify the preferred payment modes of customers whose average tenure is approximately 10 months and who have placed more than 500 orders. The AVG() function calculates the average tenure for each payment mode, while ROUND() is used to display the average values clearly. GROUP BY groups the customers according to their preferred payment mode, and HAVING filters the grouped results based on the average tenure and total order count. Since there was no exact average tenure of 10 months, the rounded results were considered, giving Debit Card (9.95 months), Credit Card (9.78 months), and E-Wallet (10.20 months), with each having more than 500 orders.



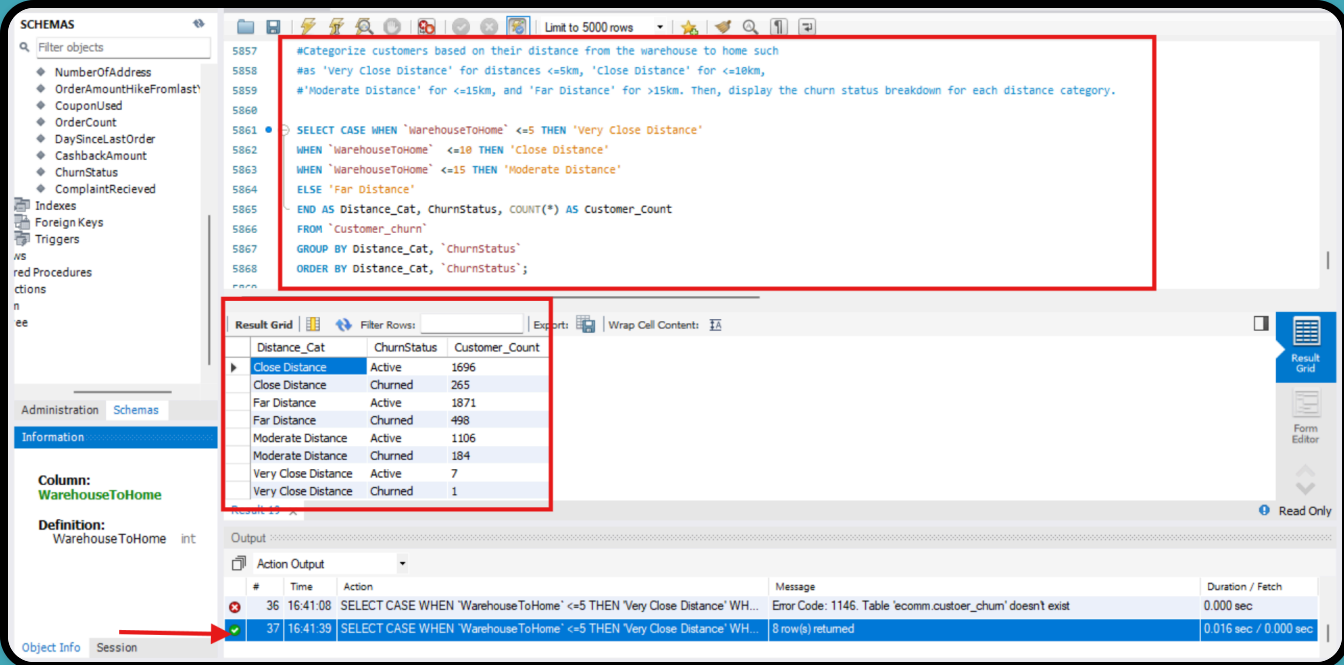
2:Comparison of Query Results

Two output screenshots are included to show the difference between the two methods. In the first method, the average tenure was checked using the exact value of 10 months, but no records were returned because none of the payment modes had an exact average tenure of 10. In the second method, the average tenure was rounded to identify payment modes with an average tenure approximately equal to 10 months. This produced the required results: Debit Card (9.95 months), Credit Card (9.78 months), and E-Wallet (10.20 months). The second method was therefore used to obtain the relevant results.



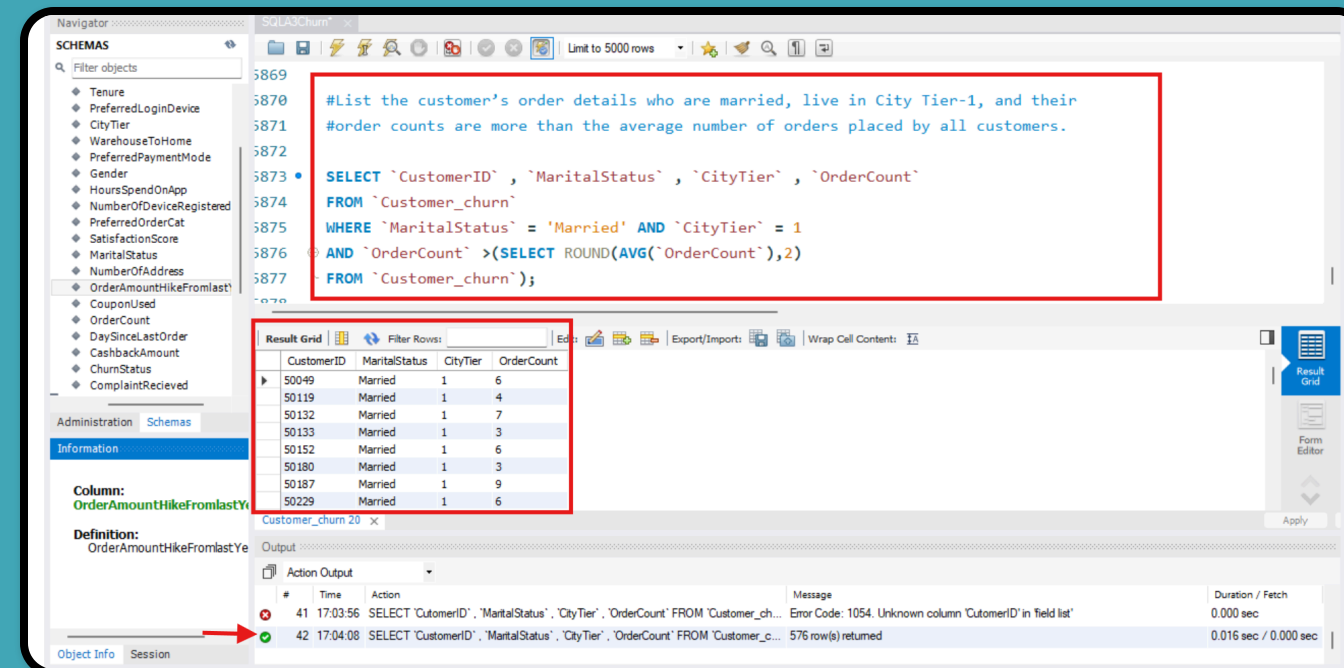
3:Customer Categorization Based on Warehouse-to-Home Distance

The query was used to categorize customers based on the distance between the warehouse and their home. The CASE statement was used to create distance categories: Very Close Distance for distances below 5 km, Close Distance for distances below 10 km, and Moderate Distance for distances below 15 km. The ELSE condition was used to classify the remaining customers as Far Distance. COUNT(*) was used to calculate the number of customers in each category, while GROUP BY grouped the records based on the distance categorization and churn status. ORDER BY was used to arrange the output for easier analysis. The result showed the customer count for each distance category and churn status, with Far Distance having the highest customer count of 1,871.



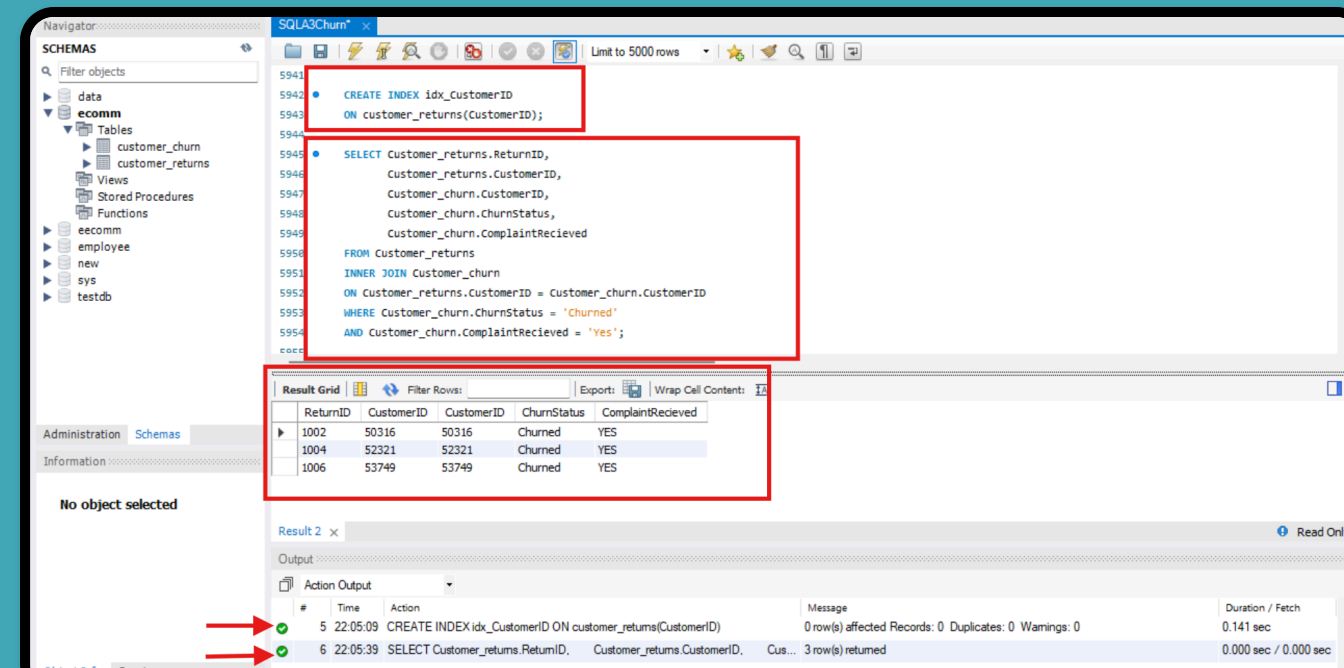
4:Customer Order Details Based on Marital Status, City Tier, and Order Count

The query was used to display the order details of customers who are married, live in City Tier 1, and have placed more orders than the average number of orders placed by all customers. The "SELECT" statement displays the Customer ID, Marital Status, City Tier, and Order Count. The "WHERE" clause applies the required conditions, while "AND" is used to combine the conditions for marital status, city tier, and order count. An inner query (subquery) was used to calculate the average "Order_Count" for all customers, and the outer query compares each customer's order count with this average. The output displays the Customer ID, marital status, City Tier 1, and order count for all customers who meet the specified conditions.



5:Creating an Index and Joining Related Tables

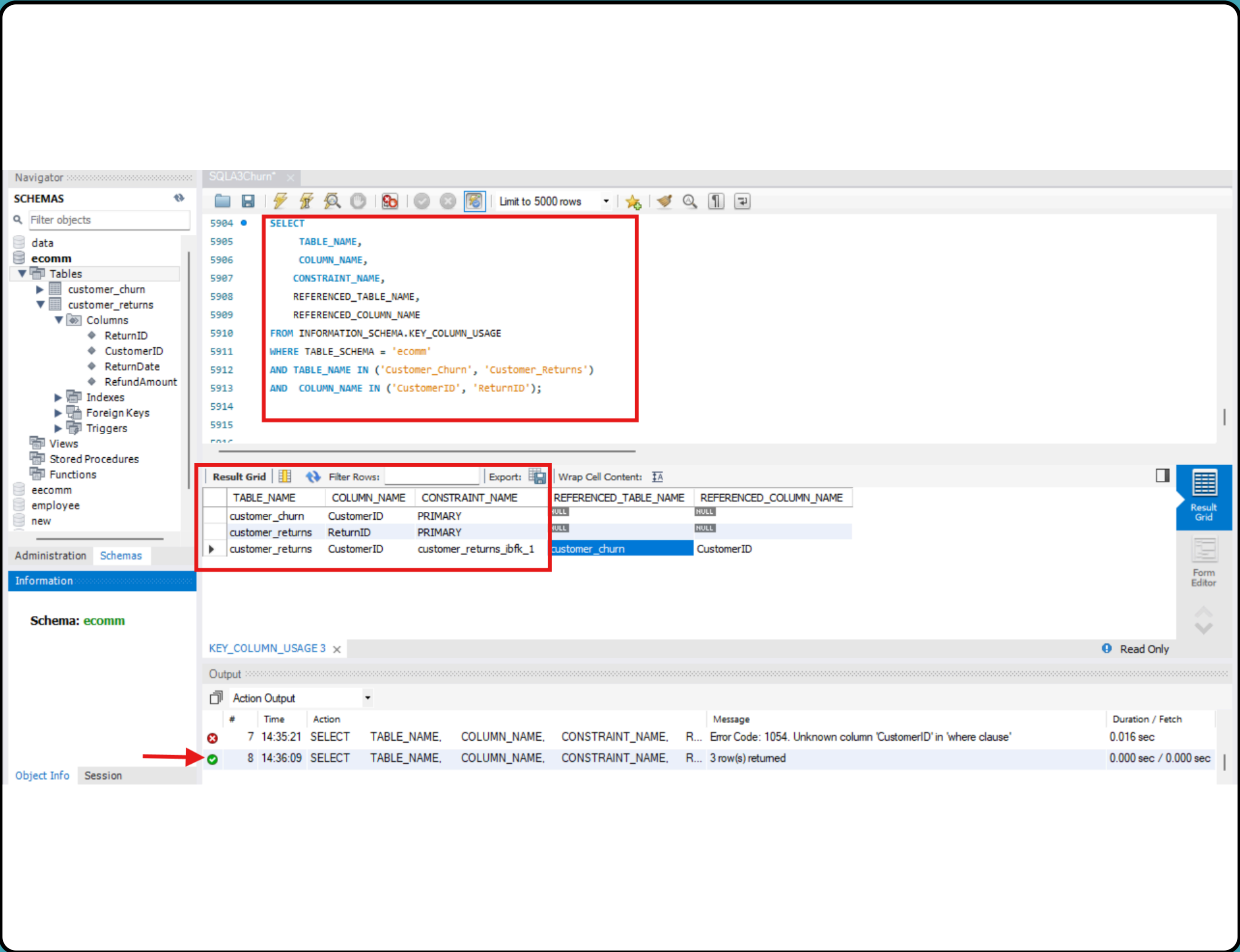
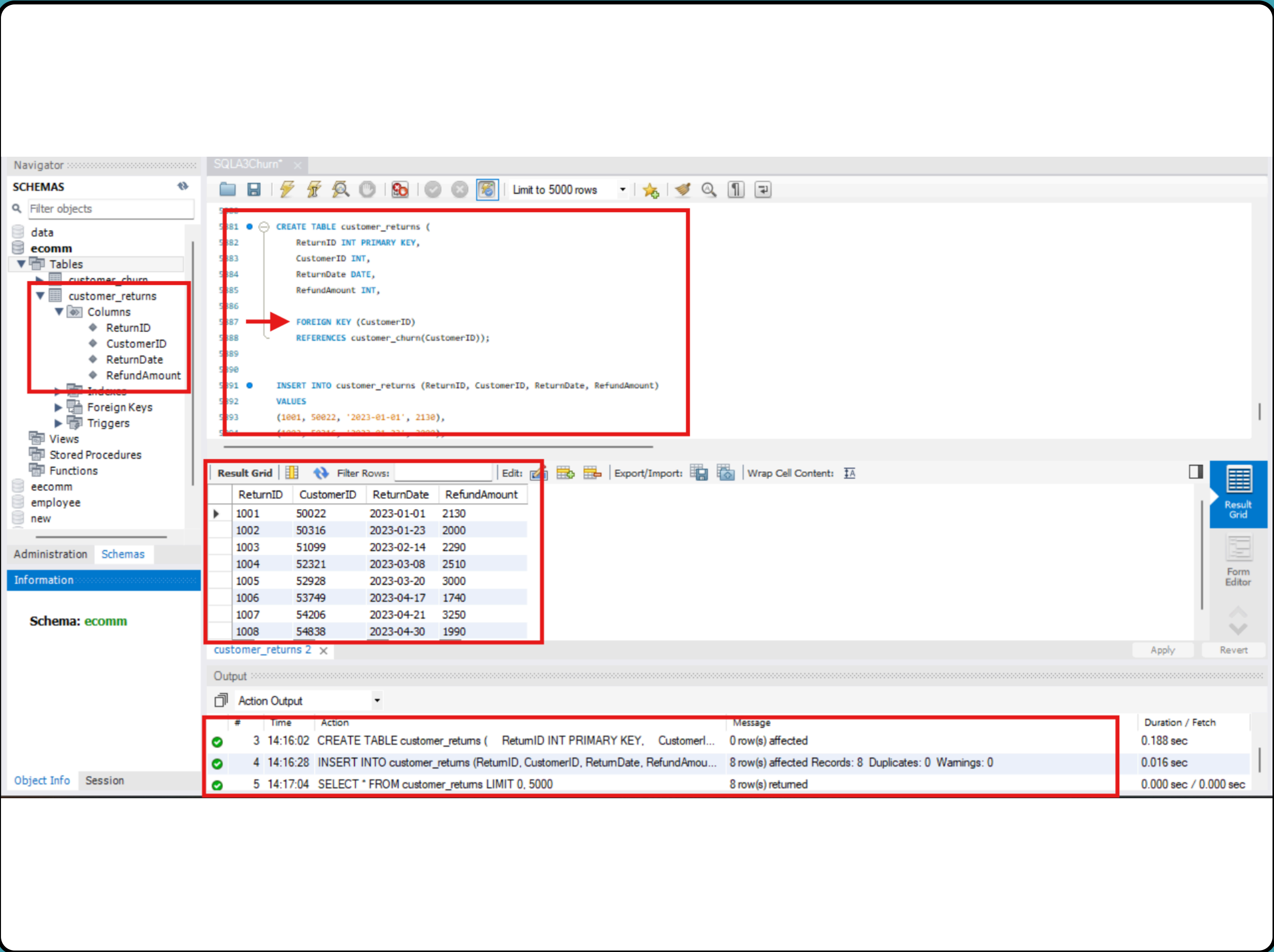
An index was created on the "CustomerID" column to improve the efficiency of searching and retrieving matching customer records. Since "CustomerID" is used as the common key between the "Customer_return" and "Customer_churn" tables, indexing this column can help improve the performance of join operations, especially when working with larger datasets. An "INNER JOIN" was then used to combine the related records from both tables and display the return details along with the customer details of customers who had churned and made complaints.



E-Commerce Customer Churn Analysis

DOCUMENTATION

Part 5- Creating New Table with Key Relationship



Creation of the Customer Returns Table

The `customer_returns` table was created in the `ecommm` database to store customer return information. The table contains four columns: `ReturnID`, `CustomerID`, `ReturnDate`, and `RefundAmount`, and eight rows of data were inserted based on the information provided in the assignment. The `ReturnID` column was used as the Primary Key to uniquely identify each return, while `CustomerID` was used as a Foreign Key to connect the `customer_returns` table with the existing `customer_churn` table, where `CustomerID` is the Primary Key. The data for the new table, including the customer IDs, return dates, and refund amounts, was provided .

The key information and relationships between both tables were retrieved using `INFORMATION_SCHEMA.KEY_COLUMN_USAGE`, which displays details about the Primary Key and Foreign Key constraints.. The result shows that `CustomerID` in the `customer_churn` table is a Primary Key, and `ReturnID` in the `customer_returns` table is also a Primary Key. It also shows that `CustomerID` in the `customer_returns` table has the constraint name `customer_returns_ibfk_1`. The term IBFK refers to the automatically generated name of the Foreign Key constraint created by MySQL, which connects `customer_returns.CustomerID` to `customer_churn.CustomerID`.

Final Transformed Customer Churn Dataset

The final transformed dataset is displayed after completing the required data cleaning, transformation, and analysis steps. The table shows the updated customer data with the necessary modifications and newly created columns. This final output represents the completed and organized dataset after applying all the required SQL operations.

SQLA3Churn

5951 AND Customer_churn.ComplaintRecieved = 'Yes';
5952
5953
5954 Select * FROM customer_churn;
5955
5956

Result Grid

	CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HoursSpendOnApp	NumberOfDeviceRegistered	PreferredOrderCat	SatisfactionScore	MaritalStatus	NumberC
	50007	1	Mobile Phone	3	11	Cash on Delivery	Male	2	3	Laptop & Accessory	2	Divorced	4
	50008	1	Mobile Phone	1	6	Credit Card	Male	3	3	Mobile Phone	2	Divorced	3
	50009	13	Mobile Phone	3	9	E wallet	Male	3	4	Mobile Phone	3	Divorced	2
	50011	4	Mobile Phone	1	18	Cash on Delivery	Female	2	3	Others	3	Divorced	2
	50014	0	Mobile Phone	1	15	Credit Card	Male	3	4	Mobile Phone	3	Divorced	1
	50020	19	Mobile Phone	1	20	Debit Card	Female	3	3	Mobile Phone	4	Divorced	10
	50021	0	Mobile Phone	3	12	Debit Card	Male	3	5	Laptop & Accessory	3	Divorced	5
	50022	20	Mobile Phone	1	29	Credit Card	Female	3	3	Fashion	2	Divorced	2
	50024	0	Mobile Phone	3	26	Debit Card	Female	3	5	Laptop & Accessory	3	Divorced	4
	50027	8	Mobile Phone	3	6	E wallet	Male	3	3	Fashion	4	Divorced	2
	50028	1	Mobile Phone	3	12	E wallet	Male	2	3	Laptop & Accessory	3	Divorced	3
	50031	2	Computer	1	6	Cash on Delivery	Male	2	3	Laptop & Accessory	3	Divorced	2
	50032	0	Mobile Phone	1	13	Credit Card	Male	2	4	Laptop & Accessory	3	Divorced	3
	50033	30	Mobile Phone	1	15	Credit Card	Female	3	4	Mobile Phone	4	Divorced	5
	50034	13	Mobile Phone	3	10	E wallet	Male	3	4	Fashion	2	Divorced	8
	50035	1	Computer	3	8	E wallet	Female	3	3	Mobile Phone	3	Divorced	7
	50036	0	Mobile Phone	1	16	Credit Card	Male	3	3	Mobile Phone	2	Divorced	2
	50038	1	Mobile Phone	1	27	Credit Card	Male	3	3	Mobile Phone	2	Divorced	2
	50043	3	Mobile Phone	3	11	Debit Card	Female	2	3	Fashion	2	Divorced	3
	50046	0	Mobile Phone	1	23	Cash on Delivery	Male	3	3	Mobile Phone	4	Divorced	1
	50056	4	Mobile Phone	1	18	Credit Card	Female	3	3	Mobile Phone	2	Divorced	9

customer_churn 5

Output

SQLA3Churn

5951 AND Customer_churn.ComplaintRecieved = 'Yes';
5952
5953
5954 Select * FROM customer_churn;
5955
5956

Result Grid

	DeviceRegistered	PreferredOrderCat	SatisfactionScore	MaritalStatus	NumberOfAddress	OrderAmountHikeFromLastYear	CouponUsed	OrderCount	DaySinceLastOrder	CashbackAmount	ChurnStatus	ComplaintRecieved
	Laptop & Accessory	2	2	Divorced	4	14	0	1	0	121	Churned	No
	Mobile Phone	2	2	Divorced	3	16	2	2	0	123	Churned	YES
	Mobile Phone	3	2	Divorced	2	14	0	1	2	127	Churned	YES
	Others	3	2	Divorced	2	16	9	15	8	295	Churned	No
	Mobile Phone	3	1	Divorced	1	17	0	1	0	134	Churned	YES
	Mobile Phone	4	10	Divorced	18	18	1	4	3	150	Churned	YES
	Laptop & Accessory	3	5	Divorced	18	6	7	7	7	162	Churned	YES
	Fashion	2	2	Divorced	12	11	15	6	6	203	Churned	No
	Laptop & Accessory	3	4	Divorced	11	1	2	2	2	146	Churned	YES
	Fashion	4	2	Divorced	13	1	1	6	5	173	Active	No
	Laptop & Accessory	3	3	Divorced	20	0	4	5	5	123	Active	YES
	Laptop & Accessory	3	2	Divorced	13	0	1	9	9	143	Active	No
	Laptop & Accessory	3	3	Divorced	17	1	1	0	0	161	Active	No
	Mobile Phone	4	5	Divorced	20	1	1	0	0	133	Active	YES
	Fashion	2	8	Divorced	11	2	2	11	11	209	Active	No
	Mobile Phone	3	7	Divorced	18	1	1	2	2	122	Active	No
	Mobile Phone	2	2	Divorced	24	0	2	2	2	123	Active	No
	Mobile Phone	2	2	Divorced	11	0	1	2	2	127	Active	No
	Fashion	2	3	Divorced	12	2	7	4	4	194	Active	No
	Mobile Phone	4	1	Divorced	16	1	1	1	1	131	Active	YES
	Mobile Phone	2	9	Divorced	23	1	1	2	2	139	Active	YES

customer_churn 5

Output

SQLA3Churn

5951 AND Customer_churn.ComplaintRecieved = 'Yes';
5952
5953
5954 Select * FROM customer_churn;
5955 SELECT * FROM customer_returns;
5956

Result Grid

	ReturnID	CustomerID	ReturnDate	RefundAmount
	1001	50022	2023-01-01	2130
	1002	50316	2023-01-23	2000
	1003	51099	2023-02-14	2290
	1004	52321	2023-03-08	2510
	1005	52928	2023-03-20	3000
	1006	53749	2023-04-17	1740
	1007	54206	2023-04-21	3250
	1008	54838	2023-04-30	1990

customer_churn 5

Output

Data Source Table			
ReturnID	CustomerID	ReturnDate	RefundAmount
1001	50022	2023-01-01	2130
1002	50316	2023-01-23	2000
1003	51099	2023-02-14	2290
1004	52321	2023-03-08	2510
1005	52928	2023-03-20	3000
1006	53749	2023-04-17	1740
1007	54206	2023-04-21	3250
1008	54838	2023-04-30	1990

CHECKING NULL VALUES

Each column in the dataset was checked individually to confirm that there were no NULL values present. Checking for NULL values is an important data-cleaning step because missing values can affect calculations, filtering, grouping, and analysis results. Verifying that all columns contain valid values helps maintain data quality, accuracy, and consistency throughout the analysis.

Navigator

SCHEMAS

Filter objects

NumberOfDeviceRegistered

PreferredOrderCat

SatisfactionScore

MaritalStatus

NumberOfAddress

OrderAmountHikeFromlastYear

CouponUsed

OrderCount

DaySinceLastOrder

CashbackAmount

ChurnStatus

ComplaintRecieved

Indexes

Foreign Keys

Triggers

customer_returns

aws

ored Procedures

ctions

Administration

Schemas

Information

Table: customer_churn

Columns:

CustomerID

Tenure

PreferredLoginDevice

CityTier

WarehouseToHome

PreferredPaymentMode

Gender

HoursSpendOnApp

Object Info

Session

SQLA3Churn*

Limit to 5000 rows

6959

6960 • SELECT * FROM customer_churn WHERE CustomerID IS NULL;

6961 • SELECT * FROM customer_churn WHERE Tenure IS NULL;

Output

Action Output

#	Time	Action	Message	Duration / Fetch
7	23:50:27	SELECT * FROM customer_churn WHERE CustomerID IS NULL LIMIT 0, 5000	0 row(s) returned	0.000 sec / 0.000 sec
8	23:52:45	SELECT * FROM customer_churn WHERE Tenure IS NULL LIMIT 0, 5000	0 row(s) returned	0.016 sec / 0.000 sec
9	23:54:05	SELECT * FROM customer_churn WHERE PreferredLoginDevice IS NULL LIM...	0 row(s) returned	0.016 sec / 0.000 sec
10	23:54:12	SELECT * FROM customer_churn WHERE CityTier IS NULL LIMIT 0, 5000	0 row(s) returned	0.016 sec / 0.000 sec
11	23:54:18	SELECT * FROM customer_churn WHERE WarehouseToHome IS NULL LIMIT ...	0 row(s) returned	0.016 sec / 0.000 sec
12	23:54:23	SELECT * FROM customer_churn WHERE PreferredPaymentMode IS NULL LIM...	0 row(s) returned	0.015 sec / 0.000 sec
13	23:54:27	SELECT * FROM customer_churn WHERE Gender IS NULL LIMIT 0, 5000	0 row(s) returned	0.000 sec / 0.000 sec
14	23:57:59	SELECT * FROM customer_churn WHERE HoursSpendOnApp IS NULL LIMIT 0...	0 row(s) returned	0.000 sec / 0.000 sec
15	23:58:04	SELECT * FROM customer_churn WHERE NumberOfDeviceRegistered IS NUL...	0 row(s) returned	0.016 sec / 0.000 sec
16	23:58:08	SELECT * FROM customer_churn WHERE PreferredOrderCat IS NULL LIMIT 0, ...	0 row(s) returned	0.015 sec / 0.000 sec
17	23:58:12	SELECT * FROM customer_churn WHERE SatisfactionScore IS NULL LIMIT 0, ...	0 row(s) returned	0.000 sec / 0.000 sec
18	23:58:19	SELECT * FROM customer_churn WHERE MaritalStatus IS NULL LIMIT 0, 5000	0 row(s) returned	0.000 sec / 0.000 sec
19	23:58:23	SELECT * FROM customer_churn WHERE NumberOfAddress IS NULL LIMIT 0, ...	0 row(s) returned	0.015 sec / 0.000 sec
20	23:58:30	SELECT * FROM customer_churn WHERE OrderAmountHikeFromlastYear IS N...	0 row(s) returned	0.000 sec / 0.000 sec
21	23:58:35	SELECT * FROM customer_churn WHERE CouponUsed IS NULL LIMIT 0, 5000	0 row(s) returned	0.015 sec / 0.000 sec
22	23:58:39	SELECT * FROM customer_churn WHERE OrderCount IS NULL LIMIT 0, 5000	0 row(s) returned	0.016 sec / 0.000 sec
23	23:58:42	SELECT * FROM customer_churn WHERE DaySinceLastOrder IS NULL LIMIT 0...	0 row(s) returned	0.000 sec / 0.000 sec
24	23:58:46	SELECT * FROM customer_churn WHERE CashbackAmount IS NULL LIMIT 0, ...	0 row(s) returned	0.000 sec / 0.000 sec
25	23:58:49	SELECT * FROM customer_churn WHERE ChumStatus IS NULL LIMIT 0, 5000	0 row(s) returned	0.000 sec / 0.000 sec
26	23:58:53	SELECT * FROM customer_churn WHERE ComplaintRecieved IS NULL LIMIT 0...	0 row(s) returned	0.016 sec / 0.000 sec

SQL FUNCTIONS AND CLAUSES USED

Throughout the E-Commerce Customer Churn Analysis project, various MySQL functions, clauses, and operations were used to clean, transform, and analyze the customer data. Functions such as "COUNT()", "SUM()", "AVG()", "MAX()", and "ROUND()" were used for calculations and summarizing data. Clauses such as "SELECT", "FROM", "WHERE", "GROUP BY", "HAVING", "ORDER BY", and "LIMIT" were used to filter, group, sort, and retrieve relevant records. "CASE" statements were used for customer categorization, while subqueries were used for comparisons with calculated values such as average order count. Created "INDEX" and "INNER JOIN" was used to combine related tables using common keys, and operations such as "ALTER", "UPDATE", "DROP", and table creation were used to modify and manage the database. These SQL concepts helped complete the data transformation and analysis effectively.

CONCLUSION

The E-Commerce Customer Churn Analysis project successfully used MySQL to clean, transform, manage, and analyze customer data.

Various SQL operations, functions, joins, subqueries, and table relationships were used to identify customer behavior, churn patterns, complaints, order activity, and other important business insights. The analysis helps in understanding factors related to customer churn and supports better data-driven decision-making for improving customer retention.